

Deploying CityVision on Hugging Face Spaces

A complete, repo-accurate guide

CityVision — urban scene semantic segmentation

July 27, 2026

This guide deploys CityVision as **two Hugging Face Spaces**:

- **API Space** — the FastAPI prediction service (carries the model weights, uses PyTorch).
- **App Space** — the Streamlit demo UI (lean, no PyTorch; calls the API over HTTP).

It covers the *why* (separation, Docker) and every concrete step: creating Spaces, git-lfs for the weights, what to push where, the port gotcha, wiring the two together, updating, and troubleshooting.

Repo references: `deploy/hf/api/Dockerfile`, `deploy/hf/api/README.md`, `deploy/hf/app/Dockerfile`, `deploy/hf/app/README.md`, `docker/*.Dockerfile`, `docker-compose.yml`, `.dockerignore`.

1. Mental model — how HF Spaces work

A Hugging Face **Space** is just a **git repository** hosted on `huggingface.co`, with a few extra rules:

1. **A `README.md` with a YAML front-matter header configures the Space.** The key fields:

```
---
title: CityVision Segmentation API
emoji: <road emoji>
colorFrom: blue
colorTo: green
sdk: docker          # build a Dockerfile (not the Gradio/Streamlit auto-SDK)
app_port: 7860        # the port HF routes public traffic to
pinned: false
---
```

2. **`sdk: docker` means HF builds the Dockerfile at the repo root** and runs the container. Whatever your Dockerfile's `CMD` starts is your app.
3. **HF exposes exactly one port: `app_port` (7860 by convention).** Your server *must* listen on it. This is why the HF Dockerfiles serve on 7860 while local `docker-compose` uses 8000/8501 (see the port gotcha).
4. **Public URLs:**
 - Space page (build logs, settings): `https://huggingface.co/spaces/<user>/<space>`
 - Direct app URL (what clients hit): `https://<user>-<space>.hf.space` (username + space, lowercased, non-alphanumeric → -).
 - Example: user DaryaEL, space cityvision-api → `https://daryael-cityvision-api.hf.space`.
5. **Files > 10 MB must be tracked with git-lfs** (our ResNet50 weight is ~157 MB).

You end up with **two independent Space repos**, each with a root-level `Dockerfile` + `README.md` and the subset of the project it needs.

2. Why separate frontend and backend?

Reason	Detail
Different dependency weight	Backend needs PyTorch + torchvision (~hundreds of MB) and the weights; frontend needs only Streamlit + requests + Pillow + numpy — <i>no torch</i> . Splitting keeps the UI image small and fast.
Independent scaling & cost	Inference is the expensive part. Put the API on paid always-on hardware (no cold starts) while the UI stays free . A monolith couldn't.
Clean contract	The frontend only makes HTTP calls (<code>/predict/image</code> , <code>/predict/classes</code> , <code>/health</code>). Any client — curl, browser, another app — uses the API identically.
No training/serving skew	API and training import the same cityvision/ package (classes, palette, model defs), so preprocessing is identical everywhere.

The frontend reaches the backend purely through the `CITYVISION_API` environment variable — locally `http://backend:8000` (compose network), in the cloud the API Space's public URL.

3. Why Docker (and why the Docker SDK on HF)?

- **Reproducibility** — the image pins exact versions (`torch==2.12.0`, `fastapi==0.136.3`, ...). "Works on my machine" becomes "works everywhere".
- **Same artifact locally and in the cloud** — validate with `docker compose up`, then deploy the same Dockerfile logic to HF.
- **Full control of the environment** — we install **CPU-only PyTorch wheels** (`--index-url https://download.pytorch.org/whl/cpu`), far smaller than the CUDA build. HF free/basic hardware is CPU anyway.
- **Portability / no lock-in** — the same images run on AWS/GCP/Azure/Render. HF is a convenient host, not a dependency.
- **HF sdk: docker** gives this control instead of HF's opinionated auto-SDKs — we choose the base image, deps, and start command.

4. Prerequisites (one-time)

1. A **Hugging Face account** — <https://huggingface.co/join>
2. **git** and **git-lfs** installed:

```
git lfs version    # should print a version; install git-lfs if not
git lfs install    # one-time, enables the lfs filters for your user
```

3. A **trained weight** at `backend/model/resnet50_best.pth` (the served model; `CITYVISION_ARCH=ResNet50-UNet`).
4. An **HF access token** with *write* permission (HF → Settings → Access Tokens → New token, role **Write**). Use it as the git password when pushing (username = your HF username).
5. (*Optional*) **Docker Desktop** to test locally first.

5. Step 0 — Sanity-check locally with docker compose

Before touching the cloud, confirm the two-service setup works end to end:

```
docker compose up --build
# open http://localhost:8501 (UI) and http://localhost:8000/docs (API)
```

This proves the backend image builds and loads `backend/model/resnet50_best.pth`; the frontend image builds (no torch) and talks to the backend via `CITYVISION_API=http://backend:8000` (set in `docker-compose.yml`). Stop with `docker compose down`.

6. Step 1 — Deploy the API Space

6.1 Create the Space (web UI)

1. <https://huggingface.co/new-space>
2. **Owner:** you. **Space name:** `cityvision-api`.
3. **SDK:** **Docker** → Blank/from scratch (not a template).
4. **Hardware:** start with **CPU basic (free)**; upgrade to paid always-on later to avoid cold starts.
5. **Visibility:** Public. Create — HF makes an (almost) empty git repo.

6.2 Populate the Space repo

The API Space needs, **at its repo root**:

```
Dockerfile      <- from deploy/hf/api/Dockerfile
README.md       <- from deploy/hf/api/README.md (has the HF front-matter)
cityvision/     <- shared package (constants + model defs)
backend/        <- FastAPI app + backend/model/resnet50_best.pth (via LFS!)
```

Why these: the API Dockerfile does `COPY cityvision/` and `COPY backend/`, sets `ENV CITYVISION_ARCH=ResNet50-UNet`, and serves `uvicorn backend.app:app` on 7860. It does *not* need `frontend/`, `data/`, `notebooks/`, `mlflow.db`.

```
# 1. Clone the empty Space repo (use your HF token as the password)
git clone https://huggingface.co/spaces/<user>/cityvision-api
cd cityvision-api

# 2. Enable LFS and TRACK THE WEIGHTS before adding them
git lfs install
git lfs track "*.pth"          # creates/updates .gitattributes

# 3. Copy the needed files from your project (adjust the source path)
cp /path/to/CityVision/deploy/hf/api/Dockerfile ./Dockerfile
cp /path/to/CityVision/deploy/hf/api/README.md ./README.md
cp -r /path/to/CityVision/cityvision ./cityvision
cp -r /path/to/CityVision/backend ./backend
ls -lh backend/model/resnet50_best.pth # ~157 MB

# 4. Commit and push
git add .gitattributes Dockerfile README.md cityvision backend
git commit -m "Deploy CityVision prediction API (Docker Space)"
git push
```

Critical ordering: run `git lfs track "*.pth"` *before* `git add`-ing the weight. Adding a >10 MB file without LFS gets the push rejected. Verify with `git lfs ls-files` — the `.pth` must be listed.

6.3 Watch the build & test

Open the Space's **Logs** tab (you'll see `pip install torch`, the COPYs, then `uvicorn ... 0.0.0.0:7860`). Then:

```
curl https://<user>-cityvision-api.hf.space/health
# {"status":"ok","arch":"ResNet50-UNet", ...}

# Swagger UI: https://<user>-cityvision-api.hf.space/docs

curl -X POST "https://<user>-cityvision-api.hf.space/predict/image?format=overlay" \
-F "file=@some_street.png" -o overlay.png
```

Note this URL — the App Space needs it.

7. The port gotcha (read this)

Environment	Backend	Frontend	Set where
Local docker-compose	8000	8501	docker/*.Dockerfile, docker-compose.yml
Hugging Face Space	7860	7860	deploy/hf/*/Dockerfile + <code>app_port</code> : 7860 in each README. md

HF routes public traffic only to `app_port` (7860). Each container serves a **single** service on 7860 — the whole reason we run two Spaces. The `EXPOSE 7860` and `--port 7860` in the HF Dockerfiles must match `app_port` in the front-matter.

8. Step 2 — Deploy the App Space

8.1 Create it

Same as before, name it `cityvision`. SDK = **Docker**, CPU basic (free) is fine.

8.2 Populate & point it at the API

At the repo root:

```
Dockerfile      <- from deploy/hf/app/Dockerfile
README.md       <- from deploy/hf/app/README.md
cityvision/     <- torch-free constants (palette/classes)
frontend/       <- Streamlit UI
data/samples/   <- committed sample images (demo works w/o dataset)
```

```
git clone https://huggingface.co/spaces/<user>/cityvision
cd cityvision
git lfs install

cp /path/to/CityVision/deploy/hf/app/Dockerfile ./Dockerfile
cp /path/to/CityVision/deploy/hf/app/README.md ./README.md
cp -r /path/to/CityVision/cityvision ./cityvision
cp -r /path/to/CityVision/frontend ./frontend
mkdir -p data && cp -r /path/to/CityVision/data/samples ./data/samples

git add .
```

```
git commit -m "Deploy CityVision demo UI (Docker Space)"
git push
```

8.3 Set the API URL

The app finds the backend via `CITYVISION_API`. Two ways:

- **Simplest:** edit the App Space's Dockerfile line `ENV CITYVISION_API=https://daryael-cityvision-api.hf.space` to your API URL, commit/push. (Already the default in `deploy/hf/app/Dockerfile`.)
- **Cleaner:** Space → Settings → Variables and secrets → add a **Variable** `CITYVISION_API = https://<user>-cityvision-api.hf.space`. Space env vars override the Dockerfile `ENV`.

Open <https://huggingface.co/spaces/<user>/cityvision>, pick an image → it calls your API and shows **real image** · **real mask** · **predicted mask**.

The frontend tolerates a sleeping API: `wait_for_api()` polls `/health` through the ~30–60 s cold-start window (503s) instead of failing.

9. git-lfs — the model weight, in detail

- HF rejects files > 10 MB unless tracked by LFS. `resnet50_best.pth` is ~157 MB.
- `git lfs track "*.pth"` writes the `.gitattributes` line `*.pth filter=lfs diff=lfs merge=lfs -text` that routes the weight through LFS. **Commit `.gitattributes` in the same commit as the weight.**
- Verify before pushing: `git lfs ls-files` must show the `.pth`.
- If you committed the weight *without* LFS: fastest fix is delete the local clone, re-clone, `git lfs track` first, then re-copy and re-commit (avoids history surgery).
- The full Cityscapes dataset is *not* deployed (gitignored + excluded in `.dockerignore`); only `data/samples/` ships in the App image.

10. Hardware, cold starts, and "always-on"

- **Free CPU basic Spaces sleep after inactivity** and take ~30–60 s to wake ("cold start"), returning 503 meanwhile.
- For a smooth demo, run the **API on paid always-on CPU** so the model stays resident. The **App** can stay free (cheap cold start; it waits for the API via `wait_for_api`).
- Change in Space → Settings → Hardware. Keep the two Spaces separate — this is the flexibility the split buys you.

11. Configuration reference (env vars)

Variable	Used by	Meaning	Default
<code>CITYVISION_API</code>	App	URL of the API Space	<code>http://backend:8000</code> (local) / your API URL (cloud)
<code>CITYVISION_ARCH</code>	API	Which architecture to serve	<code>ResNet50-UNet</code>
<code>CITYVISION_CHECKPOINT</code>	API	Explicit weight path (optional)	derived from arch
<code>CITYVISION_MAX_UPLOAD_MB</code>	API	Upload size cap	10

Set these under Space → Settings → Variables and secrets. Use a *secret* only for truly sensitive values — none here are secret.

12. Updating a deployment

HF redeploys on every push:

```
cd cityvision-api          # or cityvision
# ...make changes / drop in a newly trained weight...
git add -A && git commit -m "Update model / code" && git push
# HF rebuilds the image and restarts the container automatically.
```

To ship a **new weight**: replace `backend/model/resnet50_best.pth` in the API Space repo, commit (LFS handles it), push. No code change needed.

13. Troubleshooting

Symptom	Likely cause / fix
Build fails on push: "file too large"	Weight not tracked by LFS. <code>git lfs track "*.pth"</code> <i>before</i> adding; check <code>git lfs ls-files</code> .
Builds but "no application on port"	Server not on 7860 . Ensure <code>--port 7860</code> in CMD and <code>app_port: 7860</code> in README.md.
API /health OK but App can't connect	CITYVISION_API wrong/missing. Set it to <code>https://<user>-<space>.hf.space</code> (no trailing slash, https).
Predictions 503 for a while then recover	API cold start. Expected on free tier; <code>wait_for_api</code> retries. Use always-on to avoid.
ModuleNotFoundError: cityvision	cityvision/ not copied into the Space repo root.
Model file missing at runtime	backend/model/*.pth didn't ship (LFS pointer only, or not copied). Confirm real bytes.
Wrong model served	CITYVISION_ARCH mismatch with the weight present.

14. Deployment checklist

API Space

- ☐ Space created, SDK = Docker, port 7860
- ☐ Dockerfile (from `deploy/hf/api/`) + README.md at root
- ☐ `cityvision/` and `backend/` copied in
- ☐ `git lfs track "*.pth"` done *before* adding the weight; `.gitattributes` committed
- ☐ `backend/model/resnet50_best.pth` present as real bytes (`git lfs ls-files`)
- ☐ Build green; /health and /docs reachable
- ☐ (optional) upgraded to always-on hardware

App Space

- ☐ Space created, SDK = Docker, port 7860
- ☐ Dockerfile (from `deploy/hf/app/`) + README.md at root

- `cityvision/`, `frontend/`, `data/samples/` copied in
- `CITYVISION_API` set to the API Space URL (Dockerfile ENV or Space variable)
- UI loads and shows image · real mask · predicted mask

TL;DR

Two Docker Spaces. Each is a git repo with a root `Dockerfile` + `README.md` (front-matter, `app_port` : 7860). API Space carries `cityvision/` + `backend/` + the LFS-tracked weight and serves FastAPI on 7860. App Space carries `cityvision/` + `frontend/` + `data/samples/`, serves Streamlit on 7860, and points at the API via `CITYVISION_API`. Test locally with `docker compose up`, then `git push` each Space.